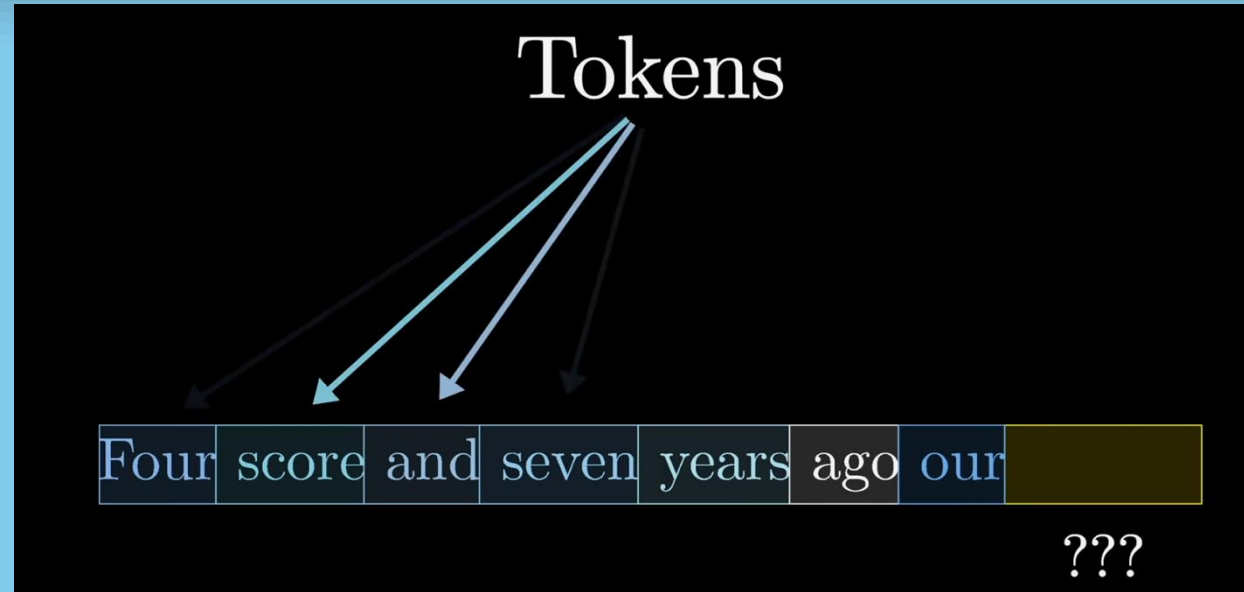
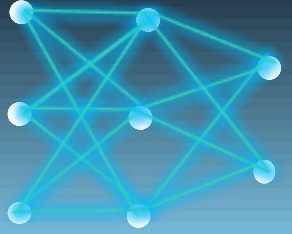




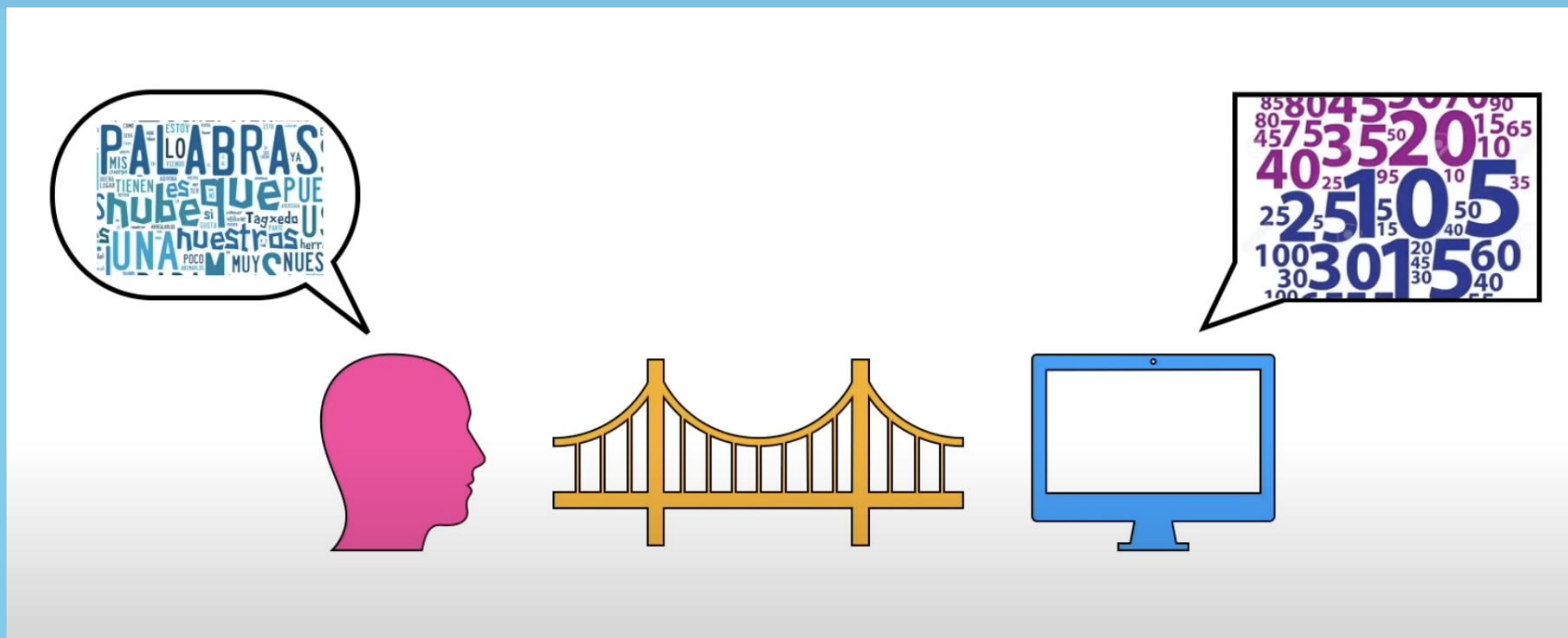
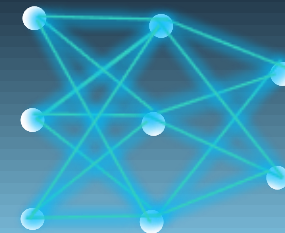
Deep Learning

Dr. Shay Maymon

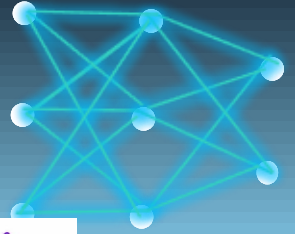
Tokenization



Word Embedding



One-hot Encoding



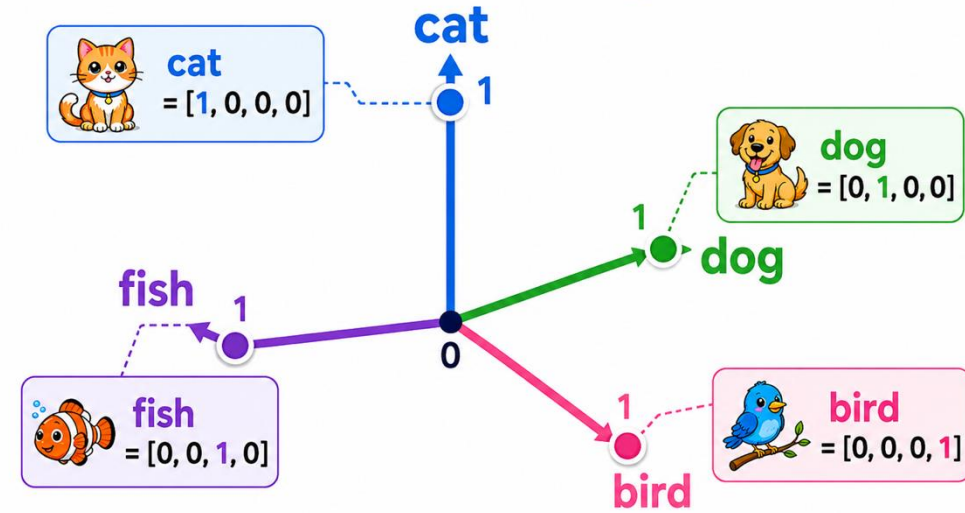
One-Hot Encoding for Word Representation

Vocabulary		One-Hot Vectors				
		1	2	3	4	
1	 cat	1	0	0	0	
2	 dog	0	1	0	0	
3	 fish	0	0	1	0	
4	 bird	0	0	0	1	



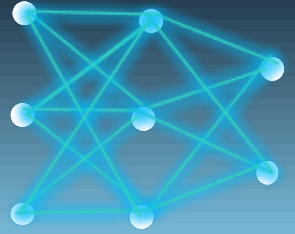
Each word is represented by a vector with one **1** and the rest **0**s.

One-Hot Encoding as Axes



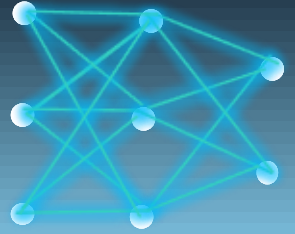
In one-hot encoding, each word gets its own axis.
The represented word has value **1** on its axis and **0** on all others.

One-hot Encoding



- There is no natural notion of **similarity** for **one-hot** vectors
- The vector representations for '**hotel**' and '**motel**' are orthogonal
- **Solution:** Learn to **encode similarity** in the **vectors themselves**

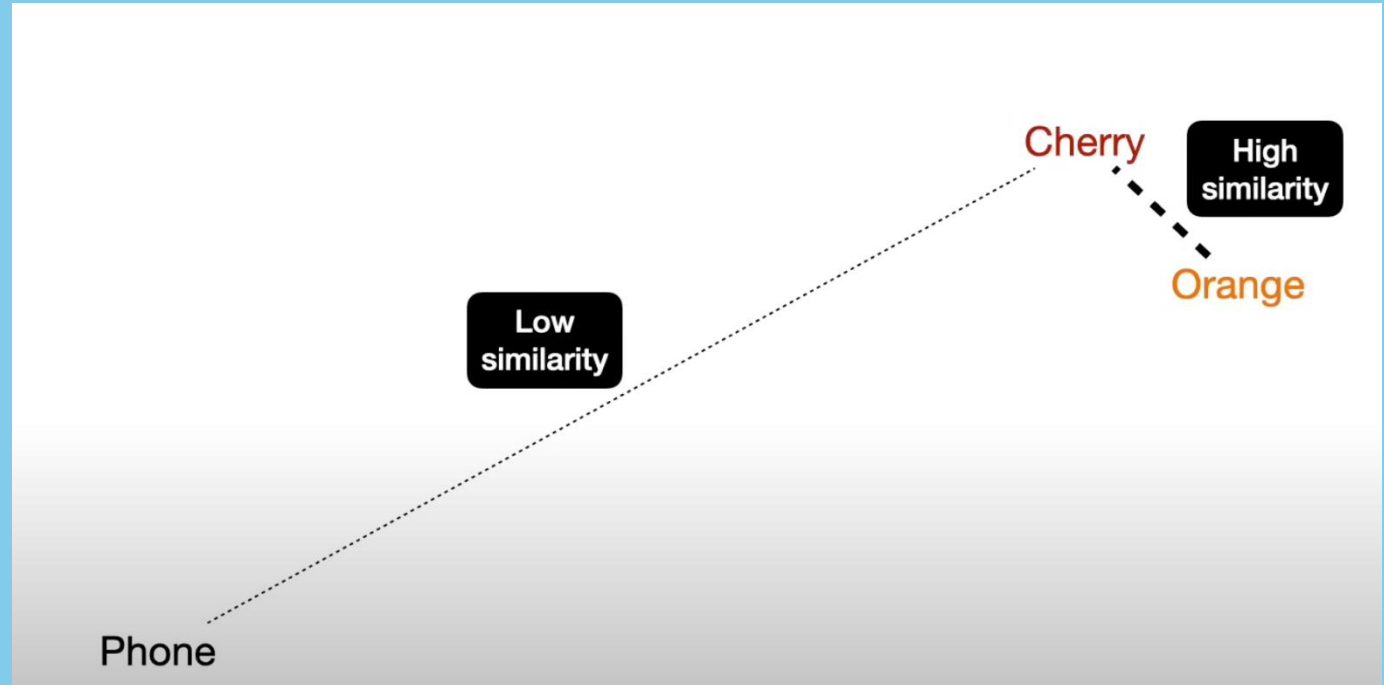
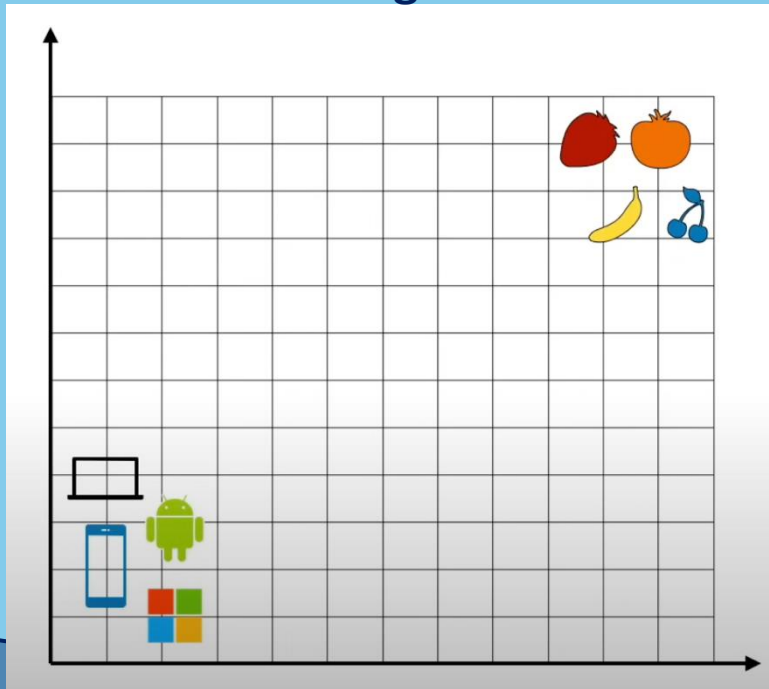
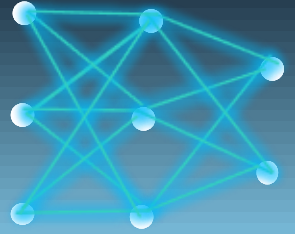
Representing Words by their Context



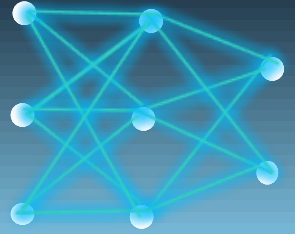
- “*You shall know a word by the company it keeps*” (J.R. Firth, 1957)
- A **word’s meaning** is given by the **words frequently appear close by**

Word Embedding

- a **numerical representation** that **encapsulates the semantic essence of a word**
- This positioning reflects the relationships and similarities between words based on their contextual **usage in vast amounts of text data**



Word Embedding

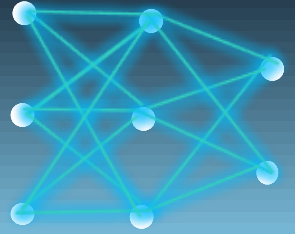


$$\mathbf{v}_n = E \mathbf{x}_n$$

$$\mathbf{v}_n \in \mathcal{R}^D \quad \mathbf{x}_n \in \mathcal{R}^{|V|}$$

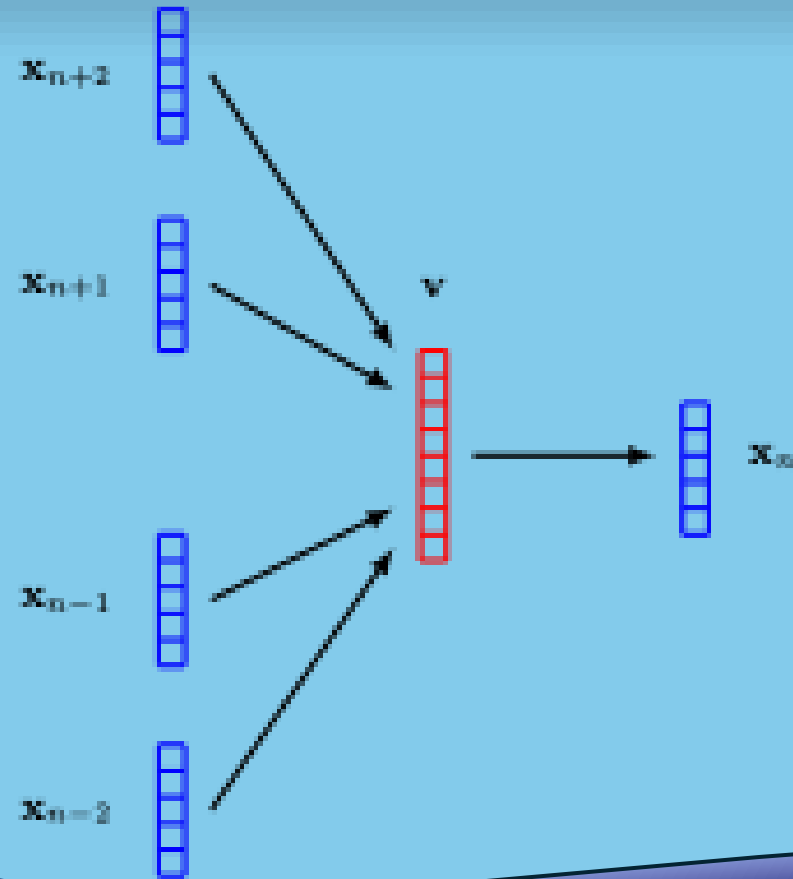
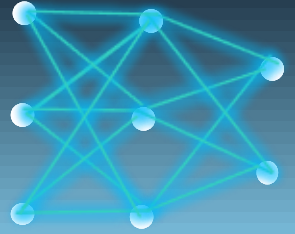
- The matrix $E \in \mathcal{R}^{D \times |V|}$ can be learned from a large data set of text

Word2Vec

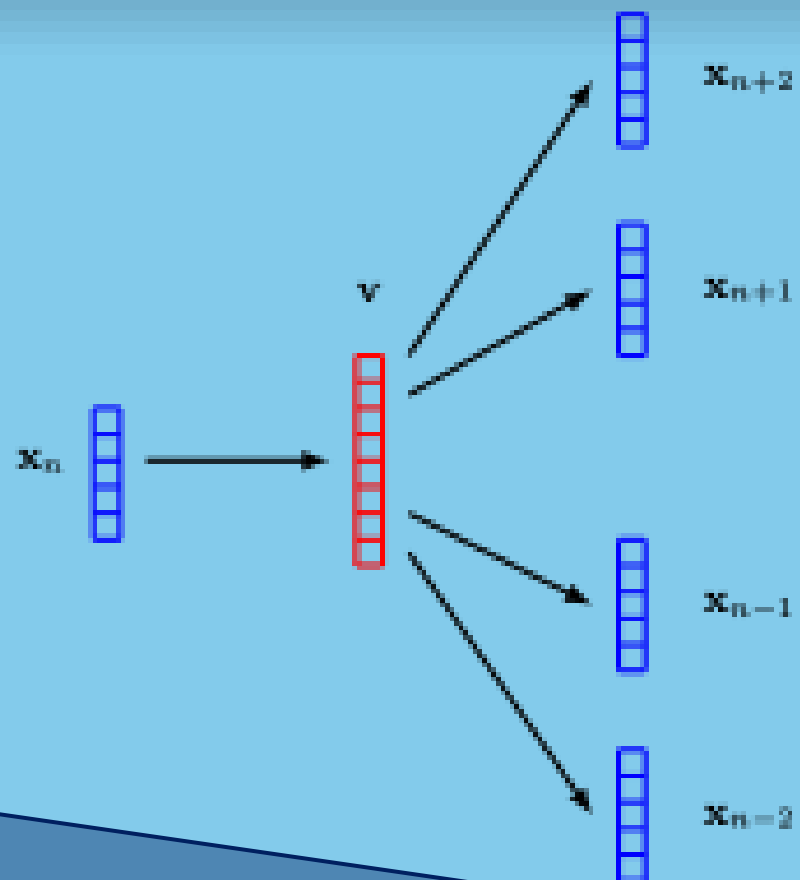
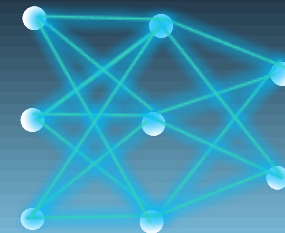


- Each sample in the training set is obtained by considering a window of M adjacent words in the text
- **Two variants:**
 - **CBOW** - Continuous Bag of Words
 - **Skip-gram**

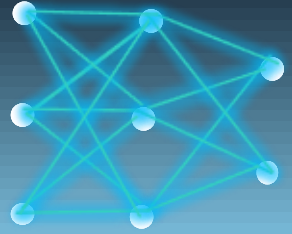
Continuous Bag of Words



Skip-grams

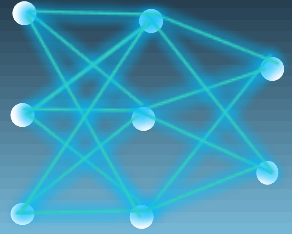


Word2Vec



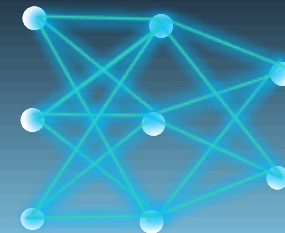
I walked across the road to get cash from the bank

Word2Vec

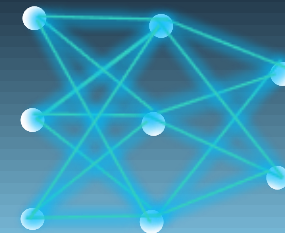


I walked across the road to get cash from the bank

Word2Vec



I walked across the road to get cash from the bank



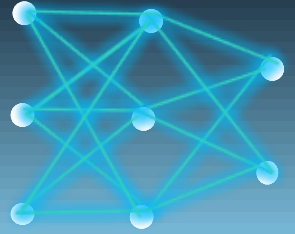
Skip-gram: Objective Function

- For each position $t = 1, 2, \dots, T$ predict context words within a window of fixed size m , given center word w_t

$$L(\theta) = \prod_{t=1}^T \prod_{-m \leq j \leq m} p(w_{t+j} | w_t; \theta)$$

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log(p(w_{t+j} | w_t; \theta))$$

Skip-gram: Objective Function



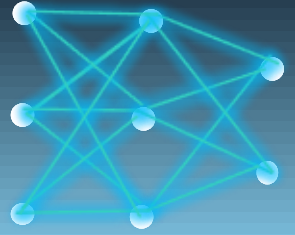
- We want to minimize the objective function:

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log \left(p(w_{t+j} | w_t; \theta) \right)$$

- How to calculate $p(w_{t+j} | w_t; \theta)$
- We will use two vectors per word w :
 - $\mathbf{v}_w$ when w is a **center** word c
 - $\mathbf{u}_w$ When w is a **context** word o

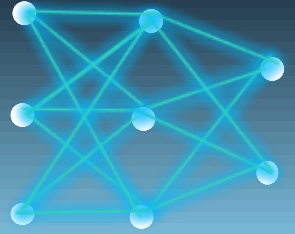
$$p(o|c) = \frac{\exp(\mathbf{u}_o^T \mathbf{v}_c)}{\sum_{w \in V} \exp(\mathbf{u}_w^T \mathbf{v}_c)}$$

Self-supervised Learning



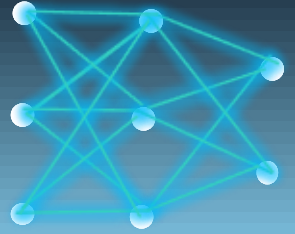
- The data consists of a large corpus of unlabelled text
- Many small windows of word sequences are drawn at random
- Labels are obtained from the text itself

The Embedding Matrix

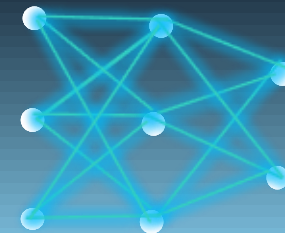


- Once the model is trained, the embedding matrix is usually obtained by the first-layer weight matrix

The embedding Space



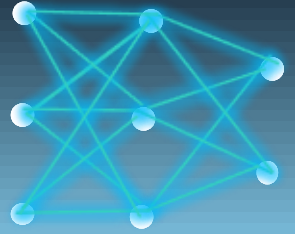
- Words that are semantically related are mapped to nearby positions in the embedding space
- Related words are more likely to occur with similar context words compared to unrelated words



The Embedding Space - Example

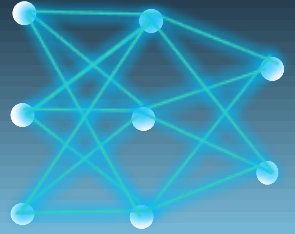
- '**city**' and '**capital**' might occur with higher frequency as context for target words '**paris**' or '**London**' and less frequently as context for '**orange**' or '**polynomial**'
- The network can more easily predict the probability of the missing words if '**Paris**' and '**London**' are mapped to nearby embedding vectors

Learned Embedding Space



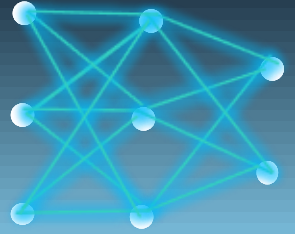
- The learned embedding space has a **richer semantic structure** than just the **proximity of related words**
- It allows for **simple vector arithmetic**

Learned Embedding Space



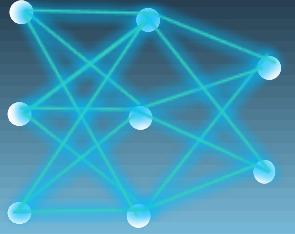
$$v(King) - v(Man) + v(Woman) \approx v(Queen)$$

Learned Embedding Space



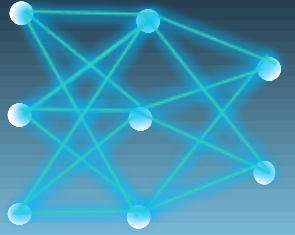
$$v(\textit{Paris}) - v(\textit{France}) + v(\textit{Italy}) \approx v(\textit{Rome})$$

Learned Embedding Space



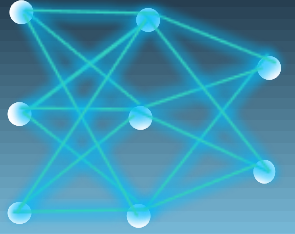
$$v(Beer) - v(Australia) + v(France) \approx v(Shampagne)$$

Learned Embedding Space



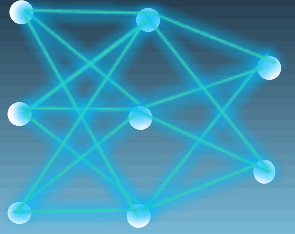
$$v(Beer) - v(Australia) + v(Russia) \approx v(Vodka)$$

Learned Embedding Space



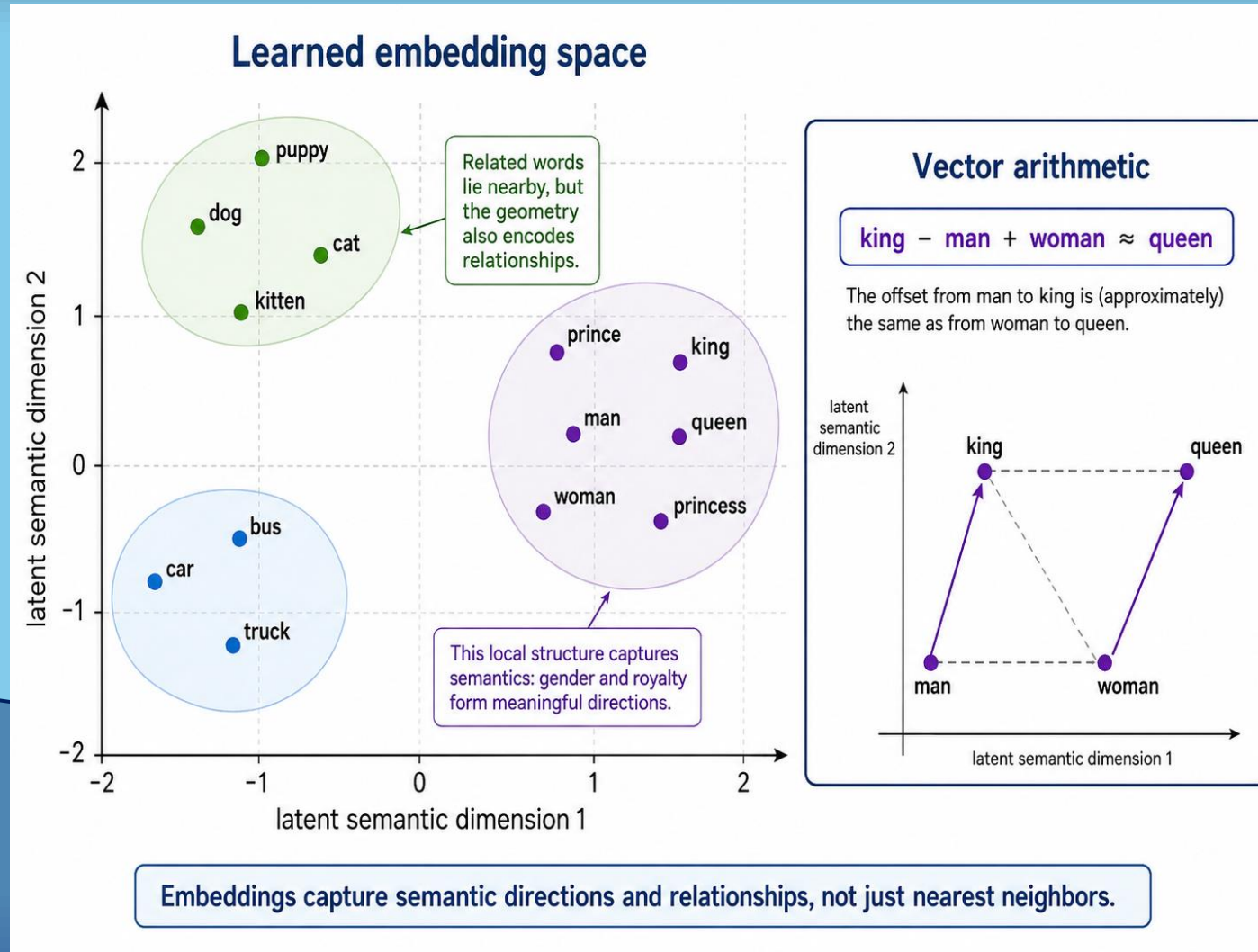
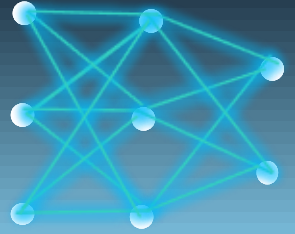
$$v(\textit{Sketching}) - v(\textit{Pencil}) + v(\textit{Camera}) \approx v(\textit{Photographing})$$

Learned Embedding Space

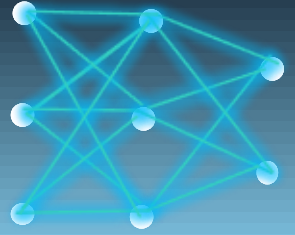


$$v(tallest) - v(tall) + v(long) \approx v(longest)$$

Learned Embedding Space

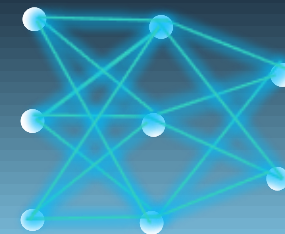


Example



I swam across the river to get to the other **bank**

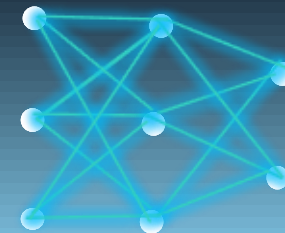
I walked across the road to get cash from the **bank**



Transformers

- One of the most important development in Deep Learning
- **Transform** a set of vectors in **some representation space** into a set of vectors in **some new space** that have a **richer internal representation**
- **Inputs**: unstructured sets of vectors, ordered sequences, or more general representations
- **Transfer learning** is very effective with transformers
- Well suited to massively prallel processing hardware such as GPUs

Attention

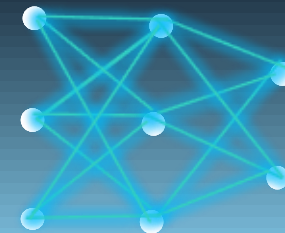


I swam across the river to get to the other **bank**

I swam across the river to get to the other bank



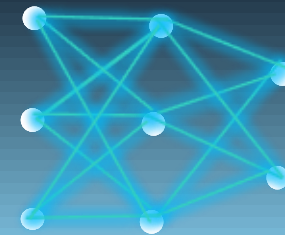
Attention



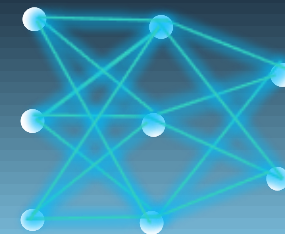
I walked across the road to get cash from the **bank**

I walked across the road to get **cash** from the **bank**

Attention

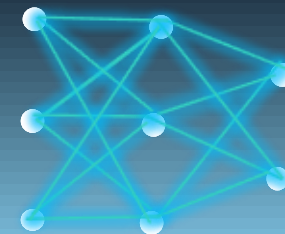


To determine the appropriate interpretation of ‘bank’, we should rely more heavily on specific words from the rest of the sentence



Attention

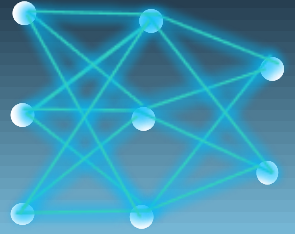
- In a **standard Neural network**, different inputs will influence the output to different extents according to the values of the weights that multiply those inputs. Once the newtwork is trained, those **weights** and their associated inputs are **fixed**
- Attention uses **weighting** factors whose values **depend** on the **specific input data**



Attention

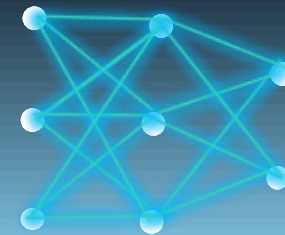
- Word embedding can be used to map words into vectors in an embedding space
- These embeddings capture elementary semantic properties, for example by mapping words with similar meanings to nearby locations in the embedding space
- **A given word always maps to the same embedding vector**
- These vectors can then be used as inputs for subsequent neural network processing

Transformer

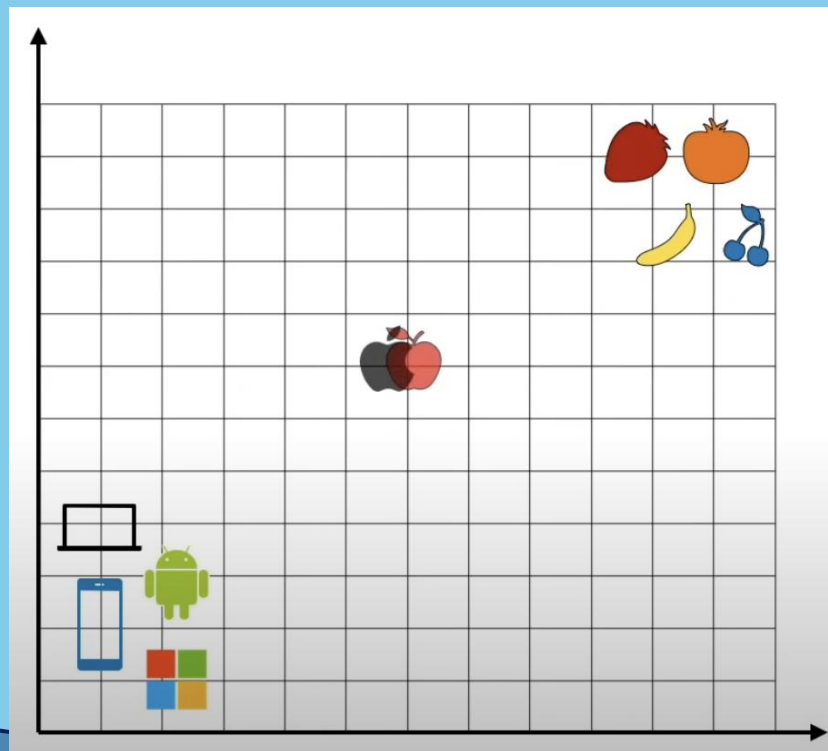


A transformer can be viewed as a **richer form of embedding** in which a given vector is mapped to a **location that depends** on the **other vectors in the sequence**

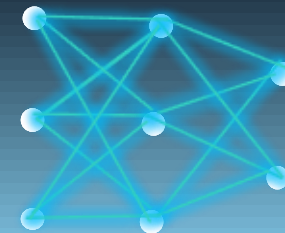
Embedding



Apple?

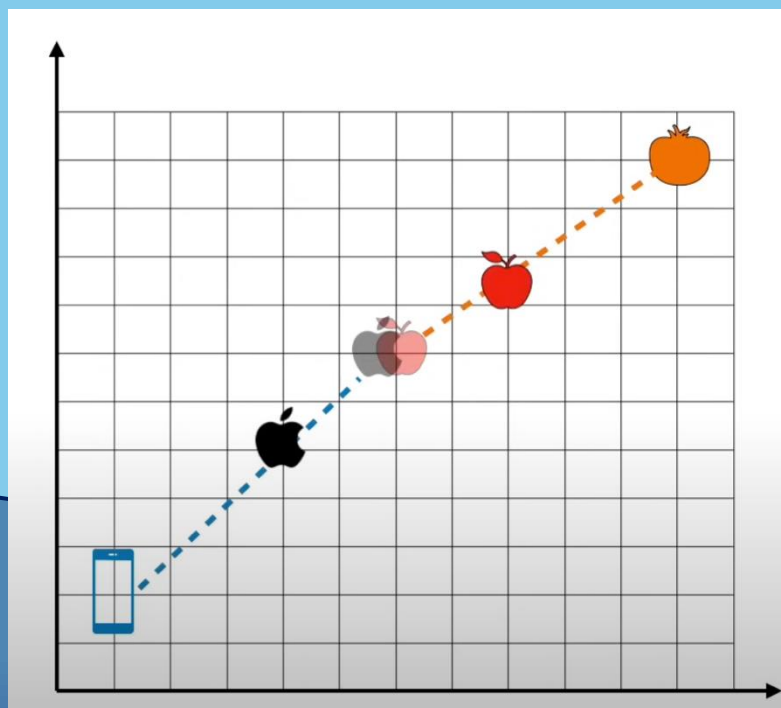


Attention

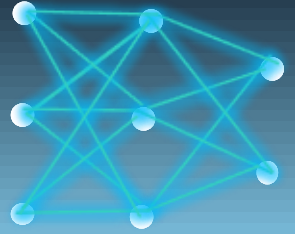


please buy an **apple** and an **orange**

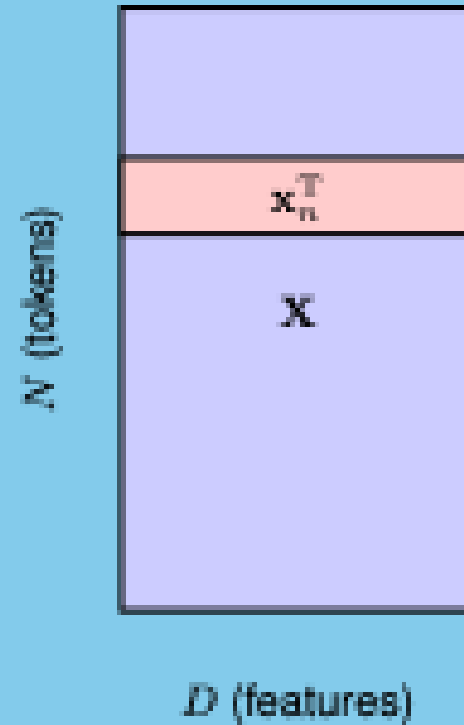
apple unveiled the new **phone**



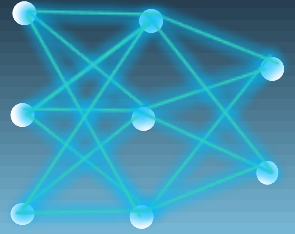
Transformer Processing



$$\tilde{X} = \text{TransformerLayer}(X)$$



Attention Coefficients



$$\mathbf{y}_n = \sum_{m=1}^N a_{nm} \mathbf{x}_m$$

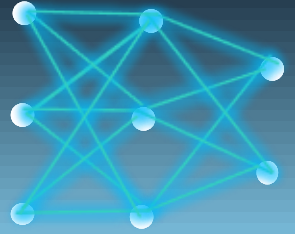
The weighting coefficients (attention weights) must satisfy the following two constraints:

$$a_{nm} \geq 0$$

$$\sum_{m=1}^N a_{nm} = 1$$

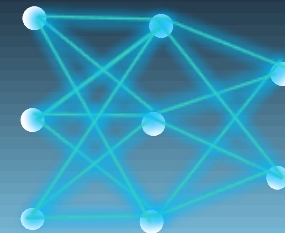
The output is a blend of the input vectors with some inputs given more weight than others

Self- Attention ('hard attention')



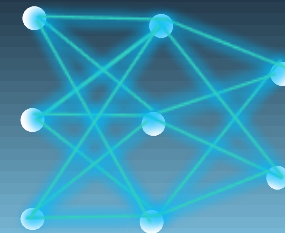
- Which movie to watch in an online movie streaming service?
 - Associate each movie with a **list of attributes** (**key**)
 - Genre (comedy, drama, action, etc.)
 - The names of the leading actors
 - The length of the movie
 - The user could provide his own personal vector of values for the **desired attributes** (**query**)
 - The corresponding movie file itself is called a **value**
 - The movie service could compare the **query** vector with all the **key** vectors to find the best match and send the corresponding movie to the user in the form of the **value** file

Key, Query, and Value



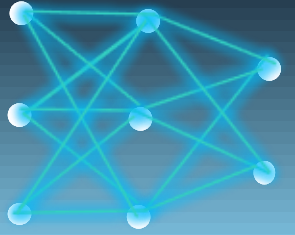
- For each word, create three vectors **key**, **query**, and **value**
- These vectors serve as instruments that determine **how much attention** a word should pay to others
- Think of the **key** vector as a **set of instructions**, the **query** vector as a **question**, and the **value** vector as the **information**

Key, Query, and Value



- **Query** – represents the entity that seeks information or attention. It is a transformed version of the input that encodes “**what to focus on?**”
- **Key** – represents the information available to compare against the query. It is a transformed version of the input that encode “**where to look?**”
- **Value** – represents the actual information corresponding to the keys. Values are derived from the input and contain the content that will be selectively attended to and used in the output

Network Parameters



$$Q = XW^{(q)} \quad V = XW^{(v)}$$

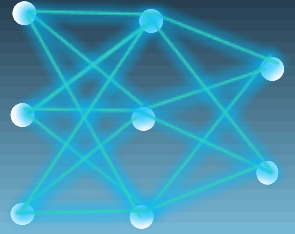
$$K = XW^{(k)}$$

$$X \in \mathcal{R}^{N \times D}$$

$$W^{(q)}, W^{(k)} \in \mathcal{R}^{D \times D_k}, D_k = D$$

$$W^{(v)} \in \mathcal{R}^{D \times D_v}$$

Dot-product Similarity



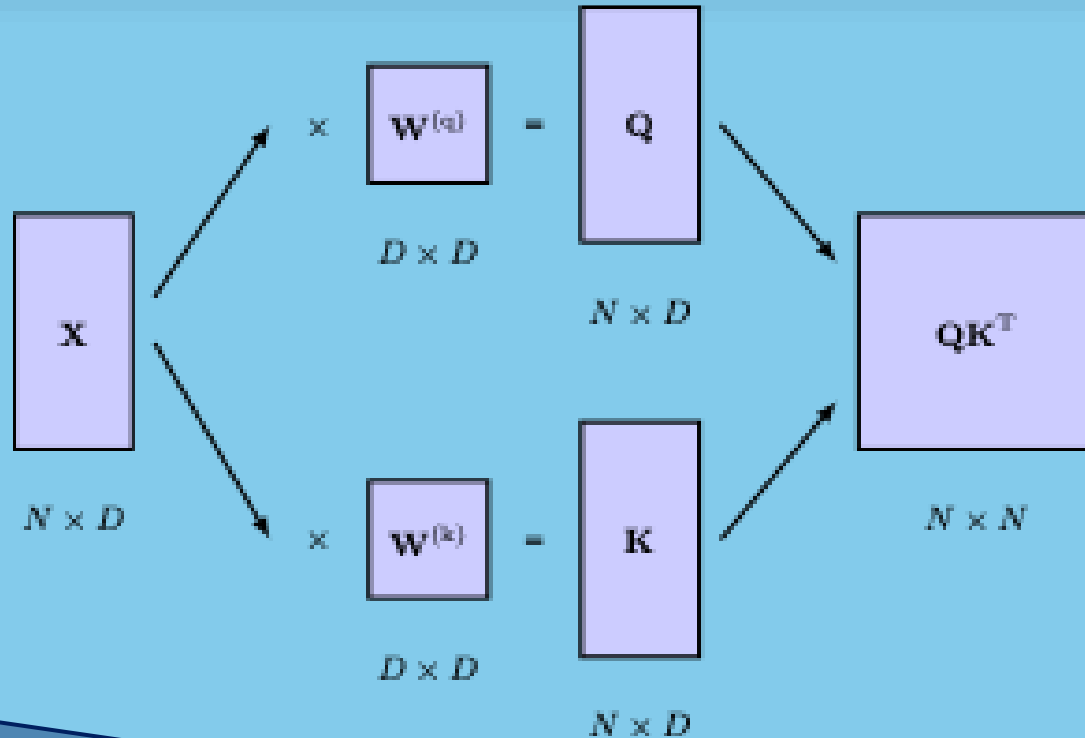
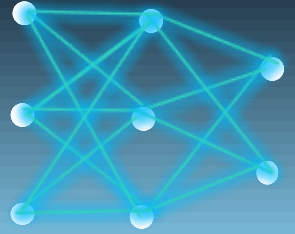
Measure 1: Dot product



		Tech	Fruitness		Tech	Fruitness
Sim	cherries	1	4	$1 \cdot 0 + 4 \cdot 3 = 12$	Tech	Fruitness
	orange	0	3			
Sim	cherries	1	4	$1 \cdot 3 + 4 \cdot 0 = 3$	Tech	Fruitness
	smartphone	3	0			
Sim	orange	0	3	$0 \cdot 3 + 3 \cdot 0 = 0$	Tech	Fruitness
	smartphone	3	0			

- **measure of similarity or correlation**
- The dot product captures **how much attention the model should assign** to each word based on their similarity in the feature space

Network Parameters



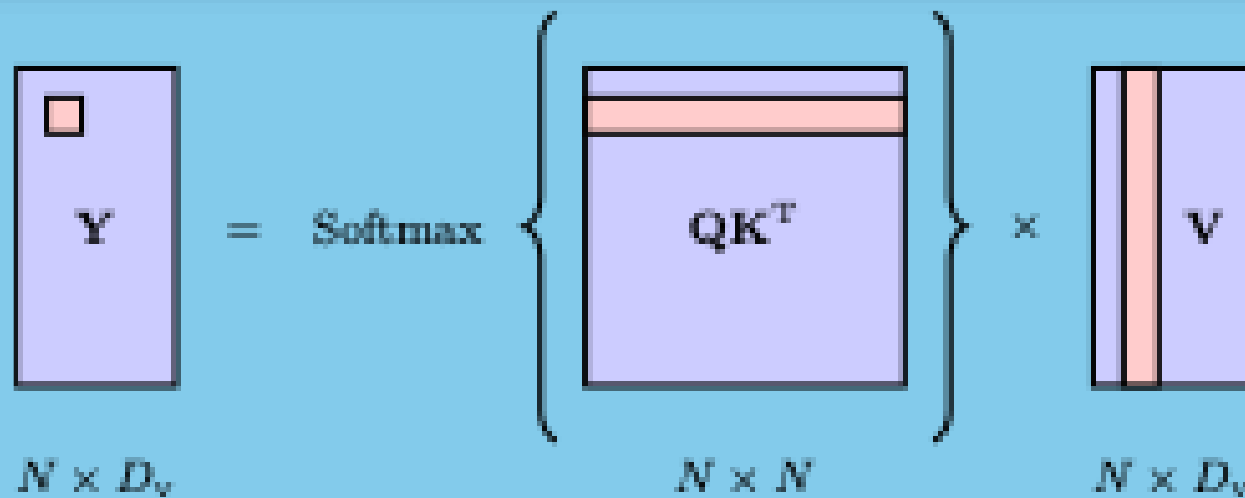
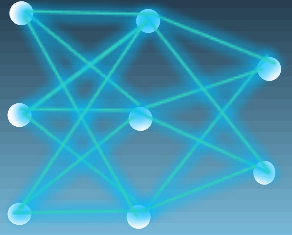
$$Q = XW^{(q)}$$

$$K = XW^{(k)} \quad X \in \mathcal{R}^{N \times D}$$

$$V = XW^{(v)}$$

$$Y = \text{Softmax}[QK^T]V$$

Network Parameters



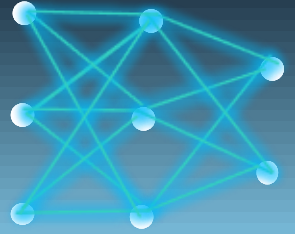
$$Q = XW^{(q)}$$

$$K = XW^{(k)} \quad X \in \mathcal{R}^{N \times D}$$

$$V = XW^{(v)}$$

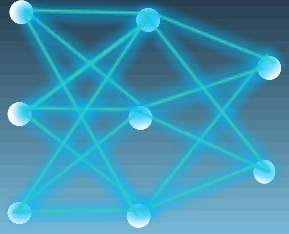
$$Y = \text{Softmax}[QK^T]V$$

Scaled Self-attention

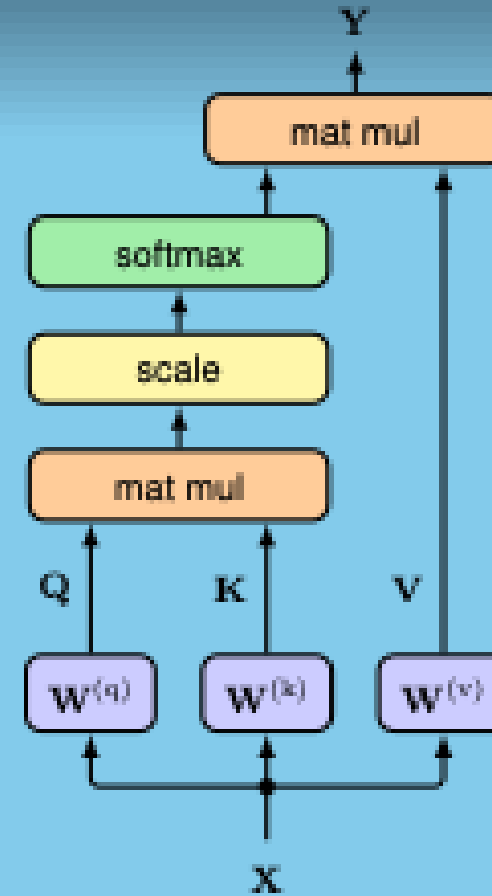


- The gradients of the softmax function become exponentially small for inputs of high magnitude
- To help prevent this from happening, we can **rescale** the product of the **query** and **key** vectors before applying the softmax function

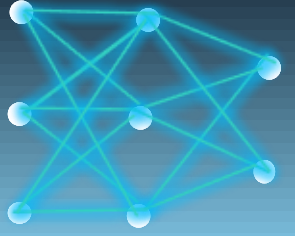
Scaled Self-attention



$$Y = \text{Attention}(Q, K, V) = \text{Softmax}\left[\frac{QK^T}{\sqrt{D_k}}\right]V$$



Multi-head Attention



- Transformers often use **multi-head attention** in which the self-attention mechanism operates **multiple times in parallel**
- This approach allows the model to capture diverse aspects and relationships within the input sequence simultaneously
- It gives the attention layer **multiple** “representation subspaces”

Multi-head Attention

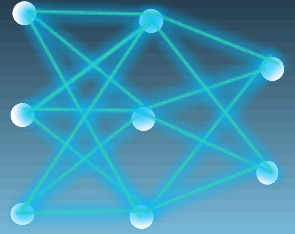
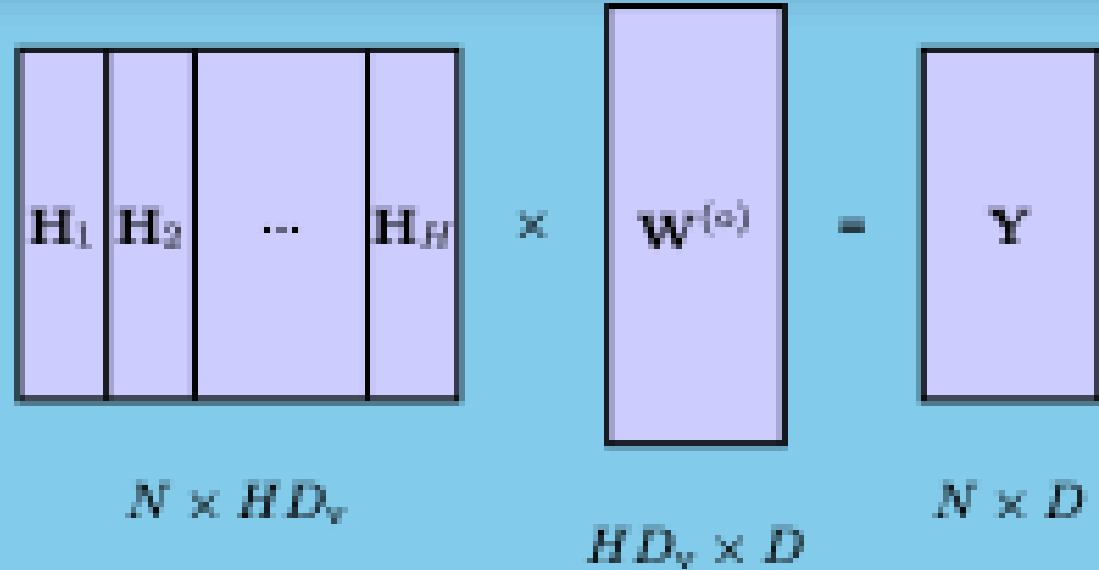
$$Q_h = XW_h^{(q)}$$

$$K_h = XW_h^{(k)}$$

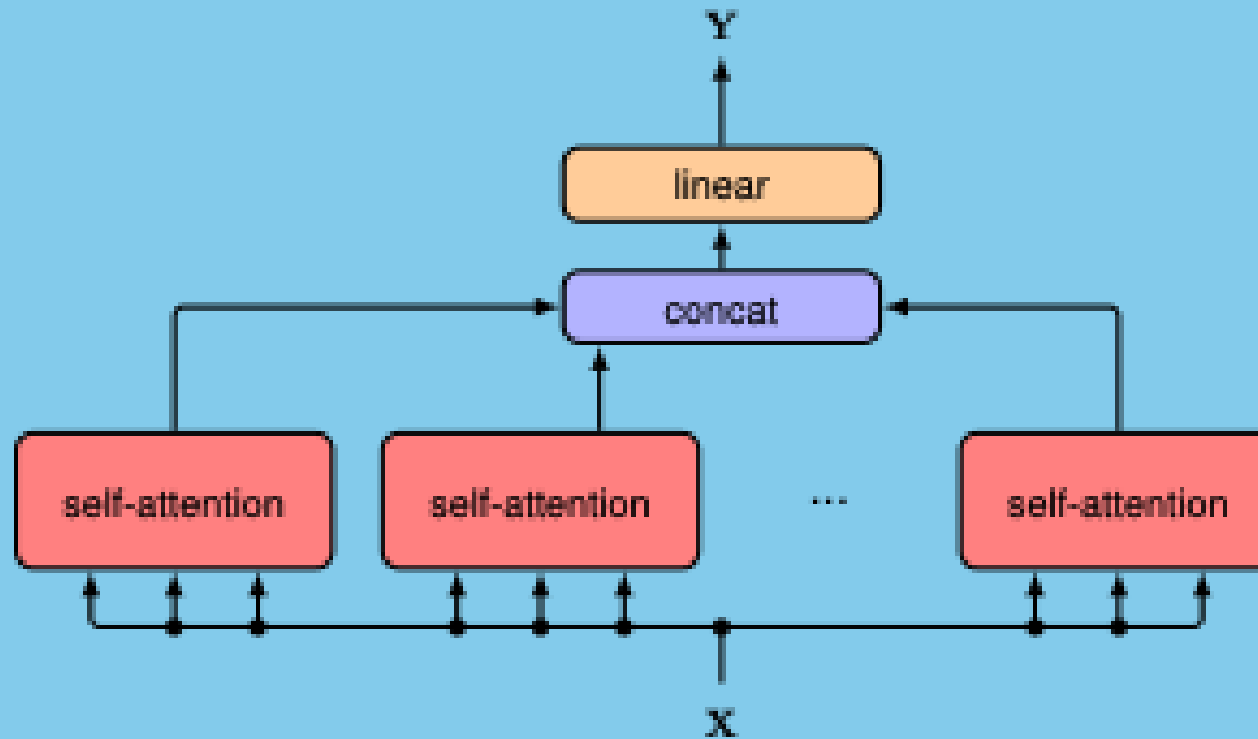
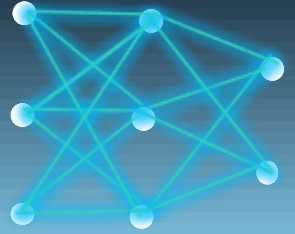
$$V_h = XW_h^{(v)}$$

$$H_h = \text{Attention}(Q_h, K_h, V_h)$$

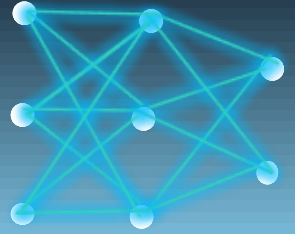
$$Y(X) = \text{Concat}[H_1, H_2, \dots, H_H]W^{(o)}$$



Multi-head Attention



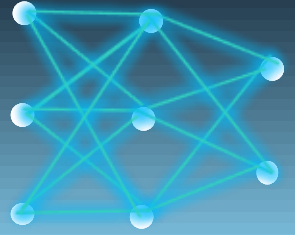
Example



The food was **bad**, not **good** at all

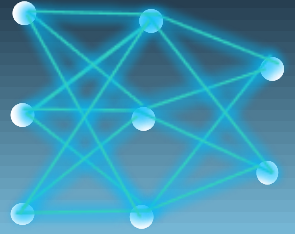
The food was **good**, not **bad** at all

Positional Encoding



- Token order is crucial for most sequential processing tasks including NLP
- We need to find a way to inject token order information into the network

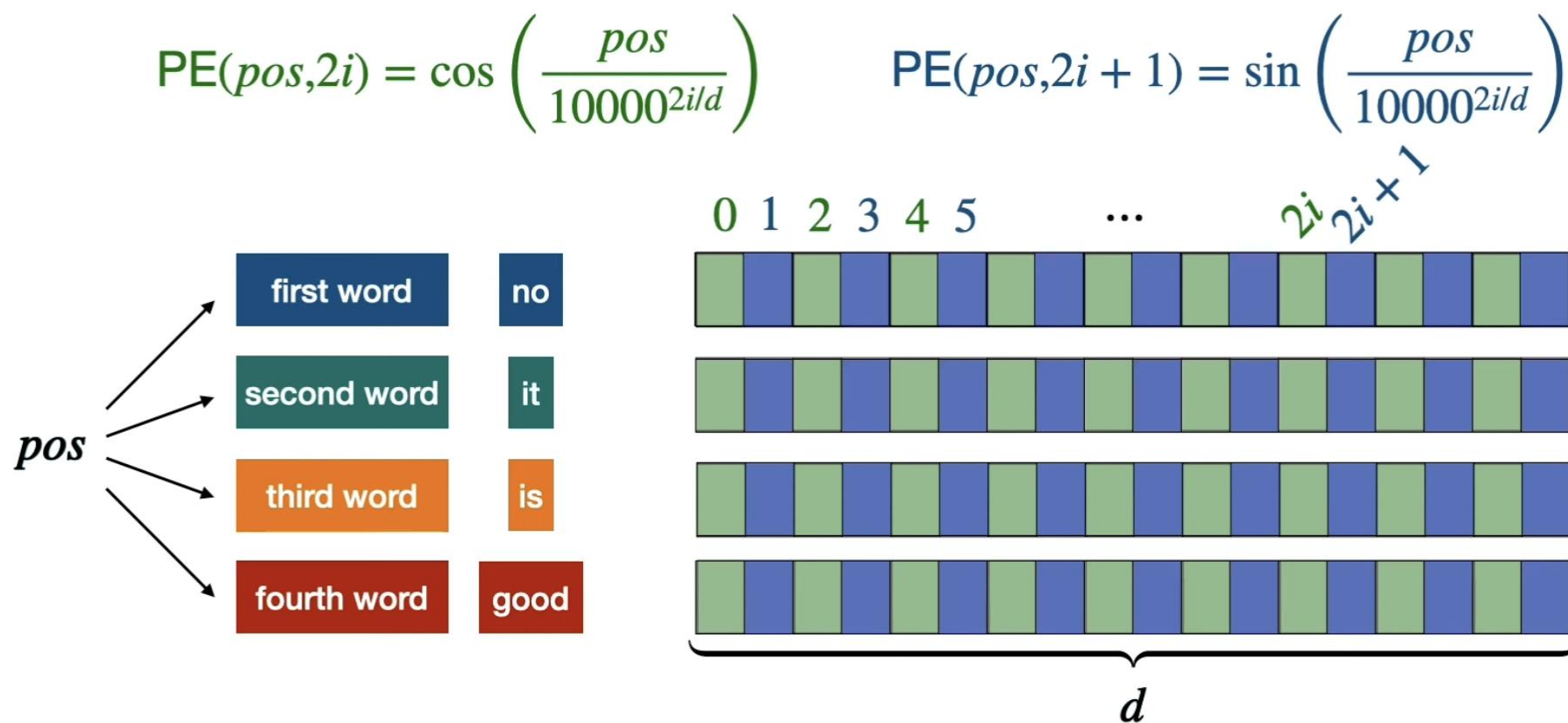
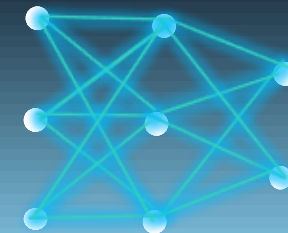
Positional Encoding



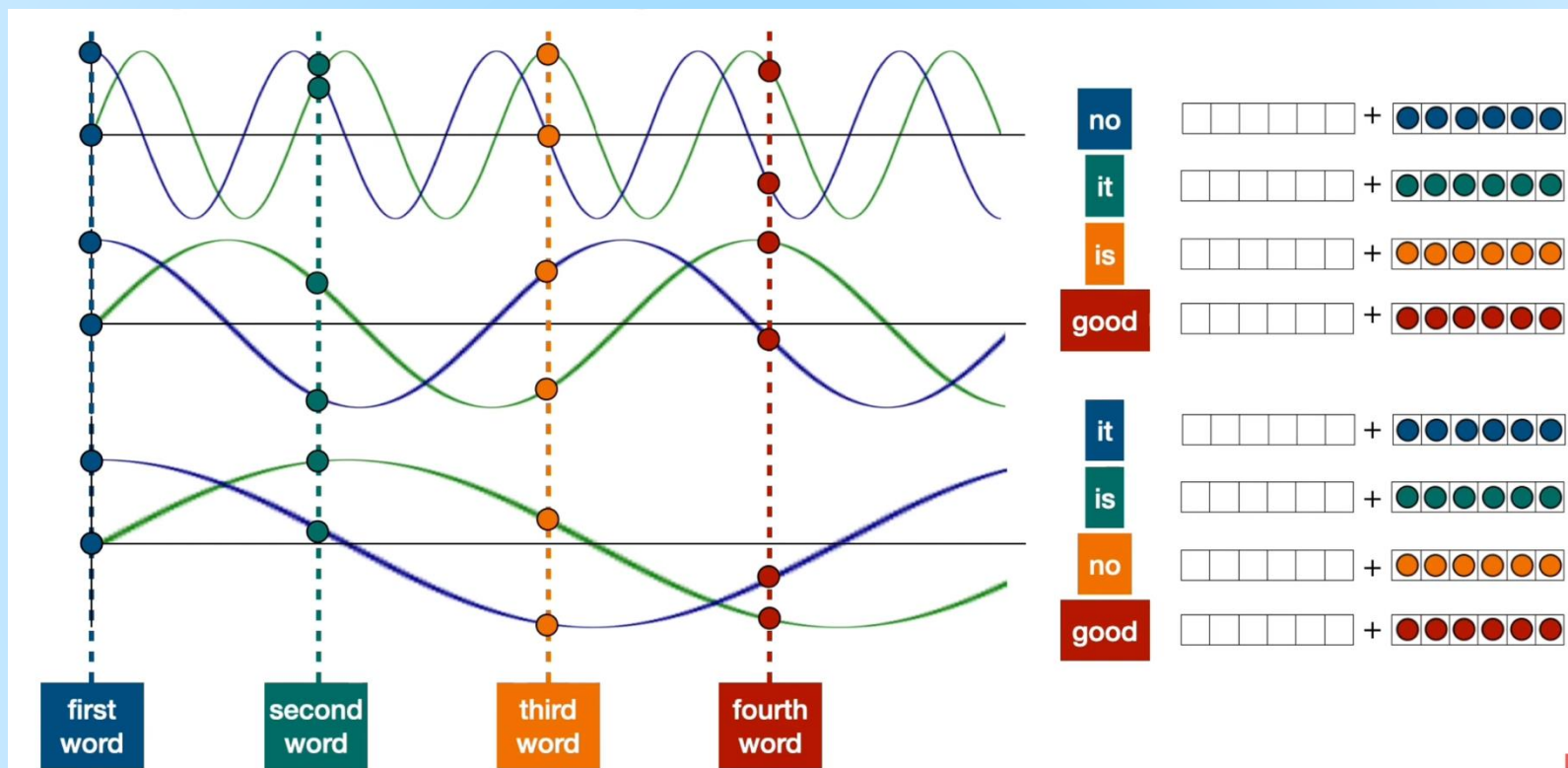
$$\tilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{r}_n$$

$$r_{ni} = \begin{cases} \sin\left(\frac{n}{L^{i/D}}\right), & \text{if } i \text{ is even} \\ \cos\left(\frac{n}{L^{(i-1)/D}}\right), & \text{if } i \text{ is odd} \end{cases}$$

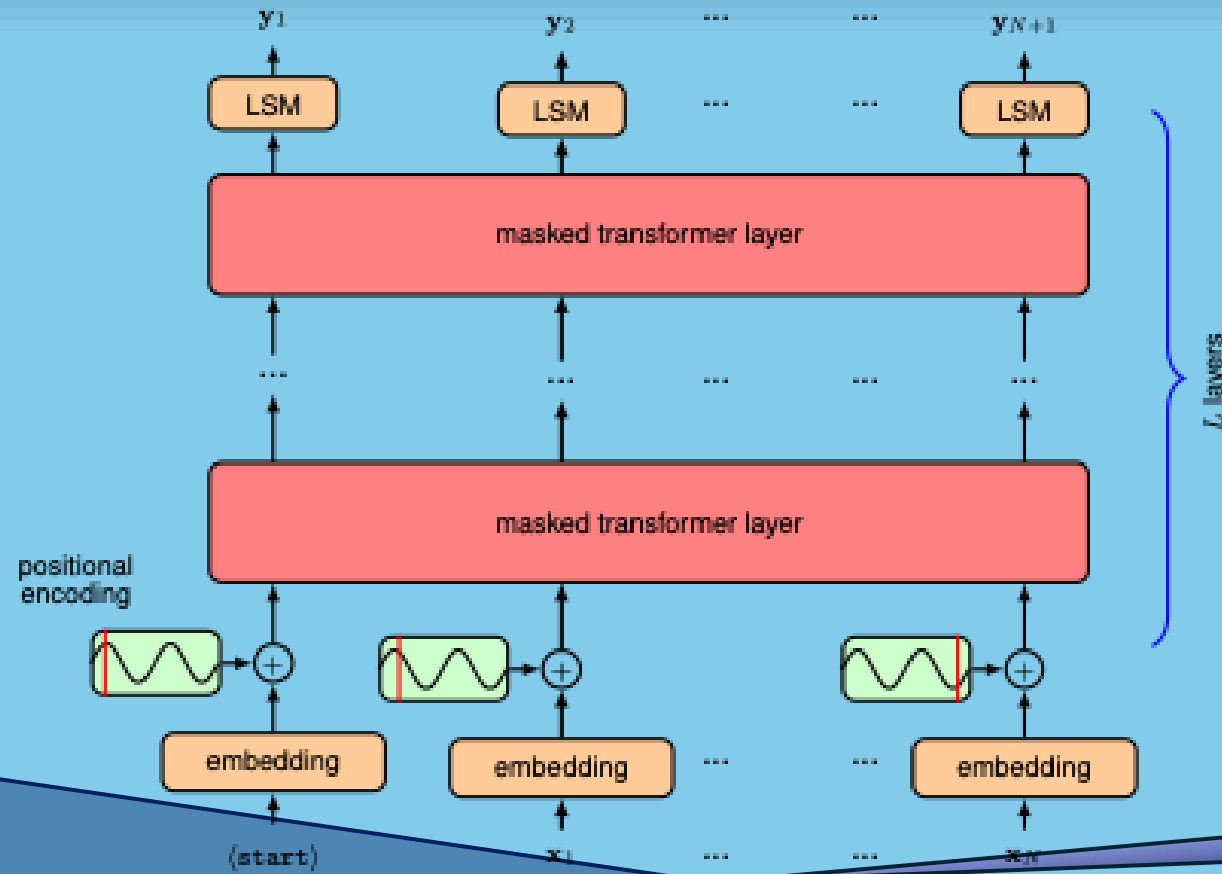
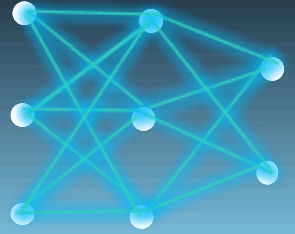
Positional Encoding



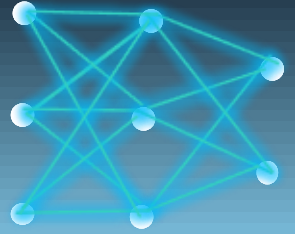
Positional Encoding



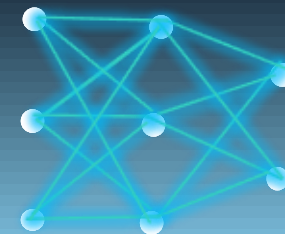
Architecture of a GPT decoder transformer network



An illustration of the mask matrix for masked self-attention

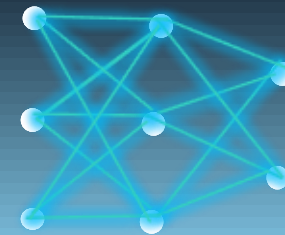


I					
swam					
across					
the					
river					
	(start)	I	swam	across	the



Course Summary

- Built a strong foundation in **Deep Learning** concepts and principles
- Learned the architecture and training of **Deep Feedforward Neural Networks**
- Understood **cost functions**, **output units**, and **hidden units**
- Explored optimization algorithms: **GD**, **SGD**, **Momentum**, **AdaGrad**, **Adam**, and **Newton's Method**
- Mastered the **Backpropagation** algorithm for neural network training
- Applied **Regularization** techniques to improve generalization and reduce overfitting
- Studied **Convolutional Neural Networks (CNNs)** for computer vision tasks
- Gained an introduction to **Natural Language Processing (NLP)**
- Learned the fundamentals of **Transformers**, the foundation of modern generative AI models



Thank you